

DTC-SYS PROJECT COMPREHENSIVE SUMMARY

1. PROJECT OVERVIEW

DTC-SYS (Document Tracking & Control System) is a comprehensive .NET-based document management and control platform designed to track, manage, and validate documents throughout their lifecycle. The system handles both internal document workflows and external vendor submissions with advanced OCR analysis, quality checking, and approval workflows.

Key Capabilities: - Physical and digital document tracking with QR code integration - Vendor submission verification with AI-powered OCR analysis - Document library management with versioning and tagging - Workflow management with multi-step approval processes - Automated SLA monitoring and alerting - Role-based access control with 5 distinct user roles - Searchable PDF generation and DPI quality validation

2. TECHNOLOGY STACK

Backend (.NET Environment)

- **Framework:** .NET 10.0
- **API Framework:** ASP.NET Core Web API
- **Database:** PostgreSQL (Supabase)
- **ORM:** Entity Framework Core 10.0.5
- **Authentication:** JWT Bearer Tokens
- **Background Jobs:** Hangfire with PostgreSQL storage
- **API Documentation:** Swagger/OpenAPI Integration
- **NuGet Packages:**
 - Hangfire.AspNetCore v1.8.23
 - Hangfire.PostgreSql v1.21.1
 - Microsoft.AspNetCore.Authentication.JwtBearer v10.0.5
 - Microsoft.EntityFrameworkCore.Design v10.0.5
 - Swashbuckle.AspNetCore v10.1.5

Python OCR Microservice

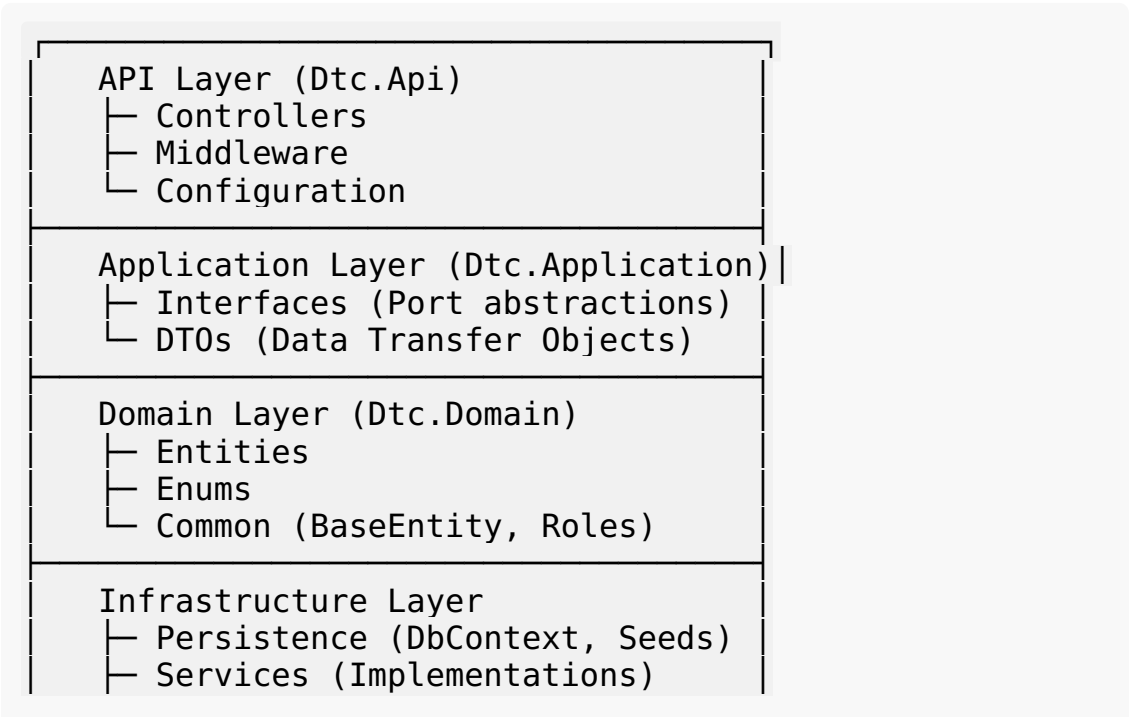
- **Framework:** FastAPI 0.115.0
- **Server:** Uvicorn 0.30.0
- **OCR Engine:** PaddleOCR 2.7.3
- **PDF Processing:** pypdfium2 4.30.0
- **Image Processing:** OpenCV, Pillow, ImageFilter, ImageEnhance
- **PDF Generation:** ReportLab 4.2.2
- **Key Libraries:**
 - paddlepaddle 2.6.2 (PaddleOCR dependency)
 - numpy 1.26.4
 - httpx 0.27.0
 - python-multipart 0.0.9

Storage & DevOps

- **File Storage:** Supabase Storage (S3-compatible)
 - **Container:** Docker-ready
 - **Database Pooler:** Supabase connection pooling
 - **Environment:** Ubuntu 24.04.3 LTS
-

3. ARCHITECTURE PATTERN: CLEAN ARCHITECTURE

The project follows **Clean Architecture** principles with four distinct layers:



- └ Jobs (Hangfire)
- └ Storage (Supabase Integration)
- └ Migrations

Architectural Benefits

- **Dependency Inversion:** High-level modules depend on abstractions
 - **Testability:** Services are mockable through interfaces
 - **Maintainability:** Clear separation of concerns
 - **Scalability:** Easy to add new features without affecting core logic
 - **Flexibility:** Easy to swap implementations (e.g., different storage providers)
-

4. DOMAIN MODEL & ENTITIES

4.1 Core Base Entity

All entities inherit from `BaseEntity`:

```
public abstract class BaseEntity
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; set; }
    public bool IsDeleted { get; set; } = false; // Soft delete support
}
```

4.2 User Entity

Represents all system users (internal staff, validators, vendors):

```
public class User : BaseEntity
{
    public string FullName { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; }
    public string Role { get; set; } // User, Admin, Validator, Vendor
    public bool IsActive { get; set; }

    // JWT Token Management
    public string? RefreshToken { get; set; }
    public DateTime? RefreshTokenExpiryTime { get; set; }
}
```

```

// Vendor-specific fields
public string? CompanyName { get; set; }
public string? ContactPhone { get; set; }

// Navigation
public ICollection<Document> Documents { get; set; }
public ICollection<WorkflowAction> WorkflowActions { get; set; }
}

```

Roles:

- SysAdmin → System administrator with full control
- Admin → Organization administrator
- Validator → Reviews vendor submissions
- User → Internal document users
- Vendor → External vendors submitting documents

4.3 Document Entity (Core)

Represents tracked documents with dual-module support:

```

public class Document : BaseEntity
{
    // Identity & Metadata
    public string DocumentNumber { get; set; } // Auto-numbered:
    public string Title { get; set; }
    public string? Description { get; set; }
    public DocumentStatus Status { get; set; }

    // Module 1: Physical Tracking
    public string? QrCode { get; set; } // QR code for ph
    public string? VendorName { get; set; }
    public string? ReferenceNumber { get; set; }
    public Guid? AssignedToUserId { get; set; }
    public DateTime? SubmittedAt { get; set; }
    public DateTime? ReceivedAt { get; set; }
    public DateTime? AssignedAt { get; set; }
    public DateTime? ReviewStartedAt { get; set; }
    public string? ReturnReason { get; set; }

    // File Storage
    public string? StoragePath { get; set; } // S3/Supabase pa
    public StorageStage StorageStage { get; set; } // Temp, Quarant
    public string? OriginalFileName { get; set; }
    public string? MimeType { get; set; }
    public long? FileSizeBytes { get; set; }

    // Module 2: Library Management
    public bool IsLibraryDocument { get; set; }
    public LibraryStatus LibraryStatus { get; set; }
}

```

```

    public string? Tags { get; set; } // Comma-separated
    public string? Category { get; set; }
    public Guid? LibraryReviewedByUserId { get; set; }
    public DateTime? LibraryApprovedAt { get; set; }
    public string? LibraryRejectionReason { get; set; }

    // Relations
    public Guid DocumentTypeId { get; set; }
    public DocumentType DocumentType { get; set; }
    public Guid CreatedByUserId { get; set; }
    public User CreatedByUser { get; set; }
    public Guid? OrganizationFunctionId { get; set; }
    public OrganizationFunction? OrganizationFunction { get; set; }

    // Collections
    public ICollection<DocumentVersion> Versions { get; set; }
    public ICollection<WorkflowInstance> WorkflowInstances { get; set; }
    public ICollection<DocumentTracking> TrackingLogs { get; set; }
}

```

4.4 Document Related Entities

DocumentType:

```

public class DocumentType : BaseEntity
{
    public string Name { get; set; }
    public string Code { get; set; }
    public string? Description { get; set; }
    public string NumberingFormat { get; set; } // e.g., "INV/{YEAR}"
    public int SequencePadding { get; set; } // Zero-padding for year
}

```

DocumentVersion:

```

public class DocumentVersion : BaseEntity
{
    public int VersionNumber { get; set; }
    public string? Notes { get; set; }
    public string? StoragePath { get; set; }
    public Guid DocumentId { get; set; }
    public Guid CreatedByUserId { get; set; }
}

```

DocumentTracking: Logs all document state changes and handover events:

```

public class DocumentTracking : BaseEntity
{

```

```

    public TrackingEvent Event { get; set; }
    public DocumentStatus? FromStatus { get; set; }
    public DocumentStatus? ToStatus { get; set; }
    public string? Notes { get; set; }
    public string? IpAddress { get; set; }

    // Handover OTP verification
    public string? OtpCode { get; set; }
    public DateTime? OtpExpiresAt { get; set; }
    public DateTime? OtpConfirmedAt { get; set; }

    // Photo proof for absent recipients
    public string? PhotoProofPath { get; set; }

    public Guid DocumentId { get; set; }
    public Guid? ActedByUserId { get; set; }
    public Guid? RecipientUserId { get; set; }
}

```

4.5 Workflow Entities

WorkflowTemplate & WorkflowStep:

```

public class WorkflowTemplate : BaseEntity
{
    public string Name { get; set; }
    public bool IsActive { get; set; }
    public Guid DocumentTypeId { get; set; }
    public ICollection<WorkflowStep> Steps { get; set; }
}

public class WorkflowStep : BaseEntity
{
    public int Order { get; set; }
    public string Name { get; set; }
    public string? Description { get; set; }
    public string RequiredRole { get; set; } // Role required
    public Guid WorkflowTemplateId { get; set; }
}

```

WorkflowInstance & WorkflowAction:

```

public class WorkflowInstance : BaseEntity
{
    public bool IsCompleted { get; set; }
    public int CurrentStepOrder { get; set; }
    public Guid DocumentId { get; set; }
    public Guid WorkflowTemplateId { get; set; }
    public ICollection<WorkflowAction> Actions { get; set; }
}

```

```

public class WorkflowAction : BaseEntity
{
    public WorkflowActionType ActionType { get; set; } // Approve, Re
    public string? Comment { get; set; }
    public int StepOrder { get; set; }
    public Guid WorkflowInstanceId { get; set; }
    public Guid ActedByUserId { get; set; }
}

```

4.6 Organization & Configuration Entities

OrganizationFunction: Represents departments/functions used in document numbering:

```

public class OrganizationFunction : BaseEntity
{
    public string Code { get; set; } // e.g., "PHR1441
    public string Name { get; set; } // e.g., "DWI Eng
    public string? Suffix { get; set; } // e.g., "S0"
    public bool IsActive { get; set; }
    public int SortOrder { get; set; }
}

```

NumberingRecord: Tracks document sequence counters:

```

public class NumberingRecord : BaseEntity
{
    public Guid DocumentTypeId { get; set; }
    public Guid? OrganizationFunctionId { get; set; }
    public int Year { get; set; }
    public string ScopeKey { get; set; }
    public string? Department { get; set; }
    public int LastSequence { get; set; }
}

```

SlaConfiguration: Defines SLA thresholds between document status transitions:

```

public class SlaConfiguration : BaseEntity
{
    public DocumentStatus FromStatus { get; set; }
    public DocumentStatus ToStatus { get; set; }
    public int MaxDurationMinutes { get; set; }
    public Guid? DocumentTypeId { get; set; } // null = applies
}

```

4.7 Vendor Submission Entity

PendingVendorRequest: Comprehensive vendor submission tracking:

```
public class PendingVendorRequest : BaseEntity
{
    // Submission Identity
    public string SubmissionNumber { get; set; }      // Auto-numbered
    public string Title { get; set; }
    public VendorSubmissionStatus Status { get; set; }

    // Vendor Information
    public string VendorCompanyName { get; set; }
    public string VendorContactName { get; set; }
    public string VendorContactEmail { get; set; }
    public string VendorContactPhone { get; set; }
    public string? ReferenceNumber { get; set; }
    public DateTime? DocumentDate { get; set; }
    public decimal? DocumentValue { get; set; }

    // File Details
    public string OriginalStoragePath { get; set; }   // Supabase path
    public string? SearchablePdfPath { get; set; }   // Searchable PDF
    public string FileName { get; set; }
    public long FileSizeBytes { get; set; }
    public int PageCount { get; set; }

    // AI Analysis Results
    public string? DetectedDocumentType { get; set; } // INVOICE, S
    public string? ExtractedFieldsJson { get; set; }  // JSON per c
    public string? DetectedSignatoryName { get; set; }
    public AiGrade AiGrade { get; set; }             // Grade, Ex
    public int? AiScore { get; set; }                 // 1-10 score
    public string? AiSummary { get; set; }
    public DateTime? AnalysisCompletedAt { get; set; }

    // DPI Quality Check
    public DpiCheckResult DpiCheckResult { get; set; } // Pass, Tool
    public int? DetectedDpi { get; set; }

    // Validator Decision
    public string? RejectionReason { get; set; }
    public RejectionCategory? RejectionCategory { get; set; }
    public DateTime? ValidatedAt { get; set; }

    // Resubmission Management
    public int ResubmissionCount { get; set; }
    public int MaxResubmissions { get; set; } = 3;
    public Guid? ParentSubmissionId { get; set; }    // Links to p
```

```

// Expiry & Conversion
public DateTime ExpiresAt { get; set; } // 30 days f
public Guid? ResultDocumentId { get; set; } // Approved b

// Relations
public Guid VendorUserId { get; set; }
public Guid DocumentTypeId { get; set; }
public Guid? ValidatorUserId { get; set; }
public Guid? SignatoryConfigId { get; set; }
}

```

4.8 Enums

DocumentStatus:

Draft, Submitted, Received, Assigned, UnderReview, Approved, Returned

TrackingEvent:

Created, Submitted, Received, Assigned, ReviewStarted, Approved, Retu
 Archived, HandoverInitiated, HandoverConfirmed, PhotoProofUploaded, F

VendorSubmissionStatus:

Pending, Analysing, Approved, Rejected, ReturnedForRevision, Approved

LibraryStatus:

None, Proposed, UnderReview, Approved, Archived, Rejected

StorageStage:

Temp, Quarantine, Archive

AiGrade:

Pending, Excellent, Good, Fair, Poor, Invalid

DpiCheckResult:

Pass (≥ 300 DPI), TooLow (< 300 DPI), Unknown

5. APPLICATION LAYER

5.1 Service Interfaces (Contracts)

IAuthService:

```
Task<AuthResponse> LoginAsync(LoginRequest request);
Task<AuthResponse> RegisterAsync(RegisterRequest request);
Task<AuthResponse?> RefreshTokenAsync(string refreshToken);
Task RevokeTokenAsync(string refreshToken);
Task<AuthResponse> RegisterVendorAsync(VendorRegisterRequest request);
```

IDocumentService:

```
Task<DocumentListResponse> GetAllAsync(int page, int pageSize, string
Task<DocumentDto?> GetByIdAsync(Guid id);
Task<DocumentDto> CreateAsync(CreateDocumentRequest request, Guid user
Task<DocumentDto?> UpdateAsync(Guid id, UpdateDocumentRequest request);
Task<bool> DeleteAsync(Guid id);
Task<DocumentDto> UploadFileAsync(Guid documentId, Stream fileStream,
Task<(Stream stream, string fileName, string contentType)?> DownloadF
Task<List<DocumentVersionDto>> GetVersionsAsync(Guid documentId);
Task<DocumentVersionDto> UploadNewVersionAsync(Guid documentId, Stream
```

IOcrService:

```
Task<OcrAnalysisResult> AnalyzeAsync(Stream pdfStream, string fileName
Task<(int dpi, bool pass)> CheckDpiAsync(Stream pdfStream, string file

// OcrAnalysisResult contains:
- Success: bool
- DocumentType: string (INVOICE, INVOICE, SPPP, etc.)
- ExtractedFieldsJson: JSON per doc type
- DetectedSignatory: string
- Grade: string
- AiScore: int (1-10)
- AiSummary: string
- AvgOcrConfidence: float
- TotalPages: int
- PagesAnalyzed: int
- DetectedDpi: int
- DpiPass: bool
- SearchablePdfBase64: string
- ErrorMessage: string
```

IStorageService:

```
Task<string> UploadAsync(string path, Stream stream, string contentType);
Task<Stream> DownloadAsync(string path);
Task DeleteAsync(string path);
```

IVendorService:

```
Task<VendorSubmissionDto> CreateSubmissionAsync(CreateVendorSubmissionDto dto);
Task<VendorSubmissionDto> GetSubmissionAsync(Guid id);
Task<List<VendorSubmissionDto>> GetVendorSubmissionsAsync(Guid vendorId);
Task<bool> DeleteSubmissionAsync(Guid id);
```

IValidatorService:

```
Task<List<VendorSubmissionDto>> GetQueueAsync();
Task<VendorSubmissionDto?> GetDetailAsync(Guid id);
Task<VendorSubmissionDto> ApproveAsync(Guid id, Guid validatorUserId, string comment);
Task<VendorSubmissionDto> RejectAsync(Guid id, Guid validatorUserId, string comment);
Task<VendorSubmissionDto> ReturnForRevisionAsync(Guid id, Guid validatorUserId, string comment);
```

ITrackingService:

```
Task<DocumentTracking> TrackEventAsync(Guid documentId, TrackingEventDto event);
Task<List<DocumentTracking>> GetTrackingLogsAsync(Guid documentId);
```

ILibraryService, IQrCodeService, IEmailService: - Library management (propose, review, approve) - QR code generation for physical tracking - SMTP email notifications

5.2 Data Transfer Objects (DTOs)

Authentication DTOs:

```
record LoginRequest(string Email, string Password);
record RegisterRequest(string FullName, string Email, string Password);
record RefreshRequest(string RefreshToken);
record AuthResponse(
    string Token,
    string RefreshToken,
    DateTime ExpiresAt,
    UserDto User
);
record UserDto(
    Guid Id, string FullName, string Email, string Role, bool IsActive
);
```

Document DTOs:

```

record DocumentDto(
    Guid Id,
    string DocumentNumber,
    string? QrCode,
    string Title,
    DocumentStatus Status,
    string? OriginalFileName,
    long? FileSizeBytes,
    Guid DocumentTypeId, string DocumentTypeCode, string DocumentTypeP
    Guid CreatedByUserId, string CreatedByUsername,
    DateTime CreatedAt, DateTime? UpdatedAt
);

record DocumentListResponse(
    int TotalCount, int Page, int PageSize,
    List<DocumentDto> Documents
);

record CreateDocumentRequest(
    string Title, string? Description,
    Guid DocumentTypeId,
    Guid? OrganizationFunctionId, string? Department
);

record DocumentVersionDto(
    Guid Id, int VersionNumber, string? Notes, string? StoragePath,
    Guid CreatedByUserId, string CreatedByUsername, DateTime CreatedAt
);

```

Vendor Submission DTOs:

```

record VendorSubmissionDto(
    Guid Id,
    string SubmissionNumber,
    string Title,
    VendorSubmissionStatus Status,
    string VendorCompanyName,
    string VendorContactName,
    string VendorContactEmail,
    string? ReferenceNumber,
    DateTime? DocumentDate,
    decimal? DocumentValue,
    string FileName,
    long FileSizeBytes,
    int PageCount,
    int? DetectedDpi,
    bool DpiPass,
    string? DetectedDocumentType,
    string? ExtractedFieldsJson,
    string? DetectedSignatoryName,

```

```

        string? AiGradeLabel,
        int? AiScore,
        string? AiSummary,
        bool AnalysisCompleted,
        string? RejectionCategoryLabel,
        string? RejectionReason,
        string? ValidatorNotes,
        DateTime? ValidatedAt,
        string? OriginalPdfUrl,
        string? SearchablePdfUrl,
        Guid? ResultDocumentId,
        string? ResultDocumentNumber,
        Guid VendorUserId,
        string VendorUserName,
        DateTime ExpiresAt,
        DateTime CreatedAt,
        DateTime? UpdatedAt,
        int ResubmissionCount
    );

    record CreateVendorSubmissionRequest(
        Guid DocumentTypeId,
        string VendorCompanyName,
        string VendorContactName,
        string VendorContactEmail,
        string VendorContactPhone,
        string? ReferenceNumber,
        DateTime? DocumentDate,
        decimal? DocumentValue,
        string? Notes
    );

    record RejectSubmissionRequest(
        RejectionCategory RejectionCategory,
        string RejectionReason
    );

```

6. INFRASTRUCTURE LAYER

6.1 Database Context (EF Core)

[DtcDbContext](#) configuration:

```

public class DtcDbContext : DbContext
{
    // DbSet for all entities
    public DbSet<User> Users
    public DbSet<DocumentType> DocumentTypes

```

```

    public DbSet<Document> Documents
    public DbSet<DocumentVersion> DocumentVersions
    public DbSet<DocumentTracking> DocumentTrackings
    public DbSet<WorkflowTemplate> WorkflowTemplates
    public DbSet<WorkflowStep> WorkflowSteps
    public DbSet<WorkflowInstance> WorkflowInstances
    public DbSet<WorkflowAction> WorkflowActions
    public DbSet<NumberingRecord> NumberingRecords
    public DbSet<SlaConfiguration> SlaConfigurations
    public DbSet<SignatoryConfig> SignatoryConfigs
    public DbSet<PendingVendorRequest> PendingVendorRequests
    public DbSet<SystemSetting> SystemSettings

    // Features enabled:
    // - Global query filters for soft delete on User, Document, DocumentVersion
    // - Cascade delete configuration for DocumentTracking
}

```

6.2 Persistence Configuration

Configurations folder contains EF Core model mappings: - Entity relationships - Shadow properties for audit trails - Default values - Constraints and indexes

6.3 Database Seeding

DbSeeder runs on application startup to: - Create default system users (Admin, Validator, Vendor) - Create default document types (INVOICE, SPPP, RELEASE_ORDER) - Create default organization functions - Configure SLA rules - Initialize system settings

6.4 Service Implementations

AuthService: - JWT token generation with configurable expiry (default 60 min) - Refresh token management with rotation - Password hashing using BCrypt/PBKDF2 - Role-based registration (internal users vs. vendors)

DocumentService: - Auto-numbering with format templates: {DocumentType}/{YEAR}/{FUNCTION}/{SEQ} - File upload with virus scanning (configurable) - Version control with automatic versioning - Pagination and filtering - Soft delete with restore capability

OcrService: HTTP client wrapper for Python FastAPI OCR service: - Sends PDF to OCR microservice at `http://localhost:8000` - Handles multipart form data uploads - Parses JSON response with extracted fields - DPI validation (≥ 300 DPI required) - 20-minute timeout for large documents

SupabaseStorageService (IStorageService): - S3-compatible Supabase Storage integration - Automatic folder structure:

`{userId}/{documentId}/filename.pdf` - Public URL generation for download - Quarantine folder for files under review - Archive folder for retained documents

TrackingService: - Logs every document state change - Records user ID, timestamp, IP address - Supports OTP-based handover verification - Photo proof upload for absent recipient scenarios

QrCodeService: - Generates unique QR codes per document - Format: DTC-TRK-`{YEAR}`-`{SEQUENCE}` - Used for physical document tracking in warehouses

VendorService: - Vendor submission CRUD operations - Expiry management (30 days default) - Resubmission tracking after rejection - Email notifications on status changes

ValidatorService: - Queue management for pending vendor submissions - Approval/rejection workflow - Return for revision workflow - Automatic document creation on approval

6.5 Background Jobs (Hangfire)

AnalysisJob: Scheduled asynchronous processing of vendor submissions:

```
public async Task ProcessAsync(Guid submissionId)
{
    // 1. Fetch submission from database
    // 2. Download PDF from Supabase Storage
    // 3. Send to OCR microservice at http://localhost:8000
    // 4. Evaluate DPI check - auto-reject if < 300 DPI
    // 5. If success: store extracted fields and AI analysis results
    // 6. Send email notification to vendor
    // 7. Mark as ready for validator review
}
```

SlAlertJob: Recurring job (configurable) that: - Scans documents in "Submitted" status - Checks elapsed time against SLA configuration - Sends alert emails if SLA threshold exceeded - Default checks: 4h (unprocessed), 8h (pending review)

Hangfire Configuration: - Use PostgreSQL for persistence (survives app restarts) - 2 worker threads (configurable for production) - Development dashboard at `/hangfire` (dev only) - Recurring job: "cleanup-expired-submissions" daily at 2:00 AM

6.6 Migrations

EF Core migrations in [Migrations](#) folder: - Enable `NpgsqlProviderOptions` for PostgreSQL compatibility - Global query filters for soft deletes - Migration commands: `bash dev migrate #`

Apply pending migrations dev migration NAME # Create new migration dev migrations-list # List all migrations

7. API LAYER

7.1 Controllers Overview

All controllers inherit from `ControllerBase` with `[ApiController]` attribute and use `[Route("api/[controller]")]`.

AuthController (/api/auth):

POST	/login	- User login (email + password)
POST	/register	- Create internal user account
POST	/register-vendor	- Vendor registration (public endpoint)
POST	/refresh	- Refresh expired JWT token
POST	/revoke	- Revoke refresh token (requires auth)
GET	/me	- Get current user profile (requires auth)

DocumentsController (/api/documents) - `[Authorize]`:

GET	/	- List documents (paginated, searchable)
GET	/ {id}	- Get document by ID
POST	/	- Create new document (auto-numbered)
PUT	/ {id}	- Update document metadata
DELETE	/ {id}	- Soft-delete document (Admin+ only)
POST	/ {id}/upload	- Upload file to document
GET	/ {id}/download	- Download document file
GET	/ {id}/versions	- List document versions
POST	/ {id}/versions/upload	- Upload new version

DocumentTypesController (/api/documenttypes) - `[Authorize]`:

GET	/	- List all document types
GET	/ {id}	- Get document type
POST	/	- Create (Admin+ only)
PUT	/ {id}	- Update (Admin+ only)
DELETE	/ {id}	- Delete (Admin+ only)

VendorController (/api/vendor) - `[Authorize]`:

POST	/submissions	- Create vendor submission
GET	/submissions/ {id}	- Get submission details
GET	/submissions	- List my submissions (for vendor)
POST	/submissions/ {id}/upload	- Upload PDF file
DELETE	/submissions/ {id}	- Delete submission

GET	/profile	- Get vendor profile
PUT	/profile	- Update vendor profile

ValidatorController (/api/validator) - [Authorize(Roles = Validator)]:

GET	/queue	- List pending submissions
GET	/review/{id}	- Get submission for review
POST	/review/{id}/approve	- Approve (creates Document)
POST	/review/{id}/reject	- Reject with reason
POST	/review/{id}/return-for-revision	- Return for vendor to fix

LibraryController (/api/library) - [Authorize]:

GET	/	- List library documents
POST	/[{id}]/propose	- Propose document for library
GET	/[{id}]/approval-queue	- Approval queue (Admin+)
POST	/[{id}]/approve	- Approve as library doc
POST	/[{id}]/reject	- Reject from library

UsersController (/api/users) - [Authorize(Roles = Admin)]:

GET	/	- List users (paginated)
GET	/[{id}]	- Get user details
POST	/	- Create user
PUT	/[{id}]	- Update user
DELETE	/[{id}]	- Delete user (soft)
POST	/[{id}]/change-password	- Change password

OrgFunctionsController (/api/orgfunctions) - [Authorize]:

GET	/	- List organization functions
GET	/[{id}]	- Get function
POST	/	- Create (Admin+ only)
PUT	/[{id}]	- Update (Admin+ only)
DELETE	/[{id}]	- Delete (Admin+ only)

SystemSettingsController (/api/settings) - [Authorize(Roles = SysAdmin)]:

GET	/all	- Get all settings
GET	/[{key}]	- Get setting by key
POST	/	- Create setting
PUT	/[{key}]	- Update setting
DELETE	/[{key}]	- Delete setting

TrackingController (/api/tracking):

GET	/document/{id}	- Get document tracking logs
POST	/handover/initiate	- Initiate physical handover
POST	/handover/confirm	- Confirm receipt (OTP)
POST	/photo-proof	- Upload drop-off photo

7.2 Global Error Handling Middleware

[ErrorHandlingMiddleware](#):

```
public class ErrorHandlingMiddleware
{
    // Maps exception types to HTTP status codes:
    // ArgumentException          → 400 Bad Request
    // InvalidOperationException → 400 Bad Request
    // UnauthorizedAccessException → 403 Forbidden
    // KeyNotFoundException      → 404 Not Found
    // All others                 → 500 Internal Server Error

    // Response format:
    // {
    //   "error": "exception message",
    //   "statusCode": 400,
    //   "timestamp": "2026-03-15T10:30:00Z"
    // }
}
```

7.3 Authentication & Authorization

JWT Configuration (appsettings.json):

```
{
  "Jwt": {
    "Key": "DtcSuperSecretKey2026!@#$$%^&*()_+ThisMustBe64BytesLongForH",
    "Issuer": "DTC-API",
    "Audience": "DTC-Client",
    "ExpiryMinutes": 60
  }
}
```

Token Claims: - `sub` (NameIdentifier): User ID (GUID) - `email`: User email - `role`: User role (User, Admin, Validator, Vendor, SysAdmin) - `exp`: Expiration timestamp

Authorization Policies: - `[Authorize]` - Requires any authenticated user - `[Authorize(Roles = "Admin,SysAdmin")]` - Role-based restrictions - No explicit policy definitions (simple role checks)

7.4 Health Check Endpoint

```
GET /health
```

Response:

```
{
  "status": "Healthy",
  "service": "DTC API",
  "timestamp": "2026-03-15T10:30:00Z"
}
```

7.5 Swagger/OpenAPI

Available at `/swagger/ui` in development: - Auto-generated from controller summaries - Includes request/response schemas - JWT bearer token authentication support - Try-it-out functionality for testing

8. PYTHON OCR MICROSERVICE

8.1 Overview

FastAPI-based microservice for intelligent document analysis using PaddleOCR and AI classification.

Location: [services/dtc-ocr](#) **Port:** 8000 (default) **API Docs:** <http://localhost:8000/docs> (Swagger)

8.2 Core Features

Document Classification: Automatic detection of document types with pattern matching: - SPPP (Surat Permintaan Proses Pembayaran) - INVOICE - RELEASE_ORDER - Custom patterns per document type

Field Extraction: Regular expressions and OCR confidence filtering to extract: - Invoice numbers, PO numbers - Vendor names - Payment amounts - Dates - Signatories

DPI Validation: - Checks image resolution/DPI from PDF metadata - Fails if < 300 DPI (configurable) - Detected DPI stored for auditing

Searchable PDF Generation: - Adds OCR-extracted text layer to PDF - Makes PDF fully searchable - Returned as Base64 for storage

AI Grading: Quality assessment based on: - OCR confidence scores (0-100%) - Document completeness - Field extraction success rate - Signatory detection

8.3 API Endpoints

POST /analyze

Input: PDF file (multipart/form-data)

Output: {

```
"success": bool,  
"document type": "INVOICE" | "SPPP" | "RELEASE_ORDER" | "UNKNOWN",  
"extracted fields": {  
  "NomorInvoice": "string",  
  "NomorPO": "string",  
  "NamaVendor": "string",  
  "NilaiInvoice": float,  
  ...  
},  
"detected signatory": "Name of signatory",  
"grade": "Excellent|Good|Fair|Poor|Invalid",  
"ai score": 8, # 1-10  
"ai summary": "Document quality assessment",  
"avg ocr confidence": 0.92,  
"total pages": 3,  
"pages analyzed": 3,  
"dpi check": {  
  "dpi": 300,  
  "pass": true  
},  
"searchable_pdf_base64": "JVBERi0xLjQKJ..."  
}
```

POST /check-dpi

Input: PDF file

Output: {

```
"dpi": 300,  
"pass": true  
}
```

8.4 OCR Processing Pipeline

1. Load PaddleOCR model (loaded once at startup)
2. Extract metadata from PDF (page count, DPI)
3. For each page:
 - a. Convert PDF page to image
 - b. Apply preprocessing (denoising, enhancement)
 - c. Run PaddleOCR for text extraction
 - d. Extract bounding boxes, confidence scores
 - e. Identify text regions (potential signatures, tables)
4. Apply document-specific regex patterns
5. Generate AI grade based on confidence scores
6. Create searchable PDF with text overlay
7. Return comprehensive analysis result

8.5 Document Type Rules

SPPP (Payment Request Letter):

```
keywords: ["Surat Permintaan Proses Pembayaran", "SPPP"]
required fields: ["NomorSurat", "Tanggal", "NilaiPembayaran"]
patterns: {
  "NomorSurat": [r"No[.\s:]+([A-Z0-9\-\-]+)", ...],
  "Tanggal": [r"(?:Tanggal|Date)[:\s]+(\d{1,2}\s+\w+\s+\d{4})", ...],
  "NilaiPembayaran": [r"Rp[.\s]*([\d.,]+)", ...]
}
```

INVOICE:

```
keywords: ["INVOICE", "Invoice"]
required fields: ["NomorInvoice", "NomorPO", "NilaiInvoice"]
patterns: {
  "NomorInvoice": [r"(Invoice|Invoice)\s*(?:No|Number)[:\s]*([\w\-\-\/\+])", ...],
  "NomorPO": [r"(3[70][00][00]\d+)", ...],
  "NilaiInvoice": [r"(?:Total|Amount)[:\s]*Rp[.\s]*([\d.,]+)", ...]
}
```

RELEASE_ORDER:

```
keywords: ["Release Order", "RELEASE ORDER"]
required fields: ["NomorRO", "Tanggal"]
patterns: {
  "NomorRO": [r"(3[70][00][00]\d+)", ...],
  "Deskripsi": [...]
}
```

8.6 Dependencies

fastapi==0.115.0	API framework
uvicorn==0.30.0	Web server
paddleocr==2.7.3	OCR engine
paddlepaddle==2.6.2	Deep learning framework (PaddleOCR dependency)
pypdfium2==4.30.0	PDF processing
pillow==10.4.0	Image processing
opencv-python-headless==4.9.0.80	Computer vision
reportlab==4.2.2	PDF generation (searchable PDFs)
numpy==1.26.4	Numerical operations
python-multipart==0.0.9	Form data parsing
httpx==0.27.0	Async HTTP client

9. CONFIGURATION & DEPLOYMENT

9.1 Application Settings

[appsettings.json](#):

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=aws-1-ap-southeast-1.pooler.supabase.co",
  },
  "Supabase": {
    "Url": "https://tblxkretjdkvbjdwujof.supabase.co",
    "ServiceKey": "REPLACE WITH SERVICE_ROLE_KEY",
    "StorageBucket": "dtc-storage"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  },
  "Jwt": {
    "Key": "DtcSuperSecretKey...",
    "Issuer": "DTC-API",
    "Audience": "DTC-Client",
    "ExpiryMinutes": 60
  },
  "Sla": {
    "UnprocessedHours": "4",
    "PendingReviewHours": "8",
    "ExpiryWarningDays": "3"
  }
}
```

[appsettings.Development.json](#): - Dev-specific overrides - Swagger enabled - Hangfire dashboard enabled - Detailed logging

9.2 Launch Settings

[launchSettings.json](#): - SelfHost profile for standalone execution - https://localhost:5001 & http://localhost:5000 - Environment: Development - ASPNETCORE_ENVIRONMENT=Development

9.3 Build & Deployment

NuGet Package Structure:

```

Dtc.slnx (Solution file)
├── src/Dtc.Api/           (Web API - HttpOnly - port 5000/5001)
├── src/Dtc.Application/   (Services & DTOs - no HTTP)
├── src/Dtc.Domain/        (Entities & Enums - no dependencies)
└── src/Dtc.Infrastructure/ (Database, EF, Services - no HTTP)

```

Project Relationships:

```

Dtc.Api
├── Dtc.Application (interfaces & DTOs)
├── Dtc.Infrastructure (service implementations)
└── Dtc.Domain (entity definitions)

Dtc.Application
└── Dtc.Domain (entity definitions)

Dtc.Infrastructure
├── Dtc.Application (interfaces)
└── Dtc.Domain (entities)

Dtc.Domain
(no dependencies)

```

Development Commands:

[dev.sh](#) - Development CLI tool:

```

./dev up           # Start API on http://localhost:5000
./dev build        # Build entire solution
./dev migrate      # Apply pending EF migrations
./dev migration NAME # Create new migration
./dev migrations-list # List all migrations
./dev doctor       # Check .NET version and environment

```

9.4 Docker Support

Project is Docker-ready:

```

# Suggested multi-stage build:
# Stage 1: Build .NET app
# Stage 2: Runtime image
# Environment: Use environment variables for Supabase credentials
# Ports: 5000/5001 for API, 8000 for OCR service (separate container)

```

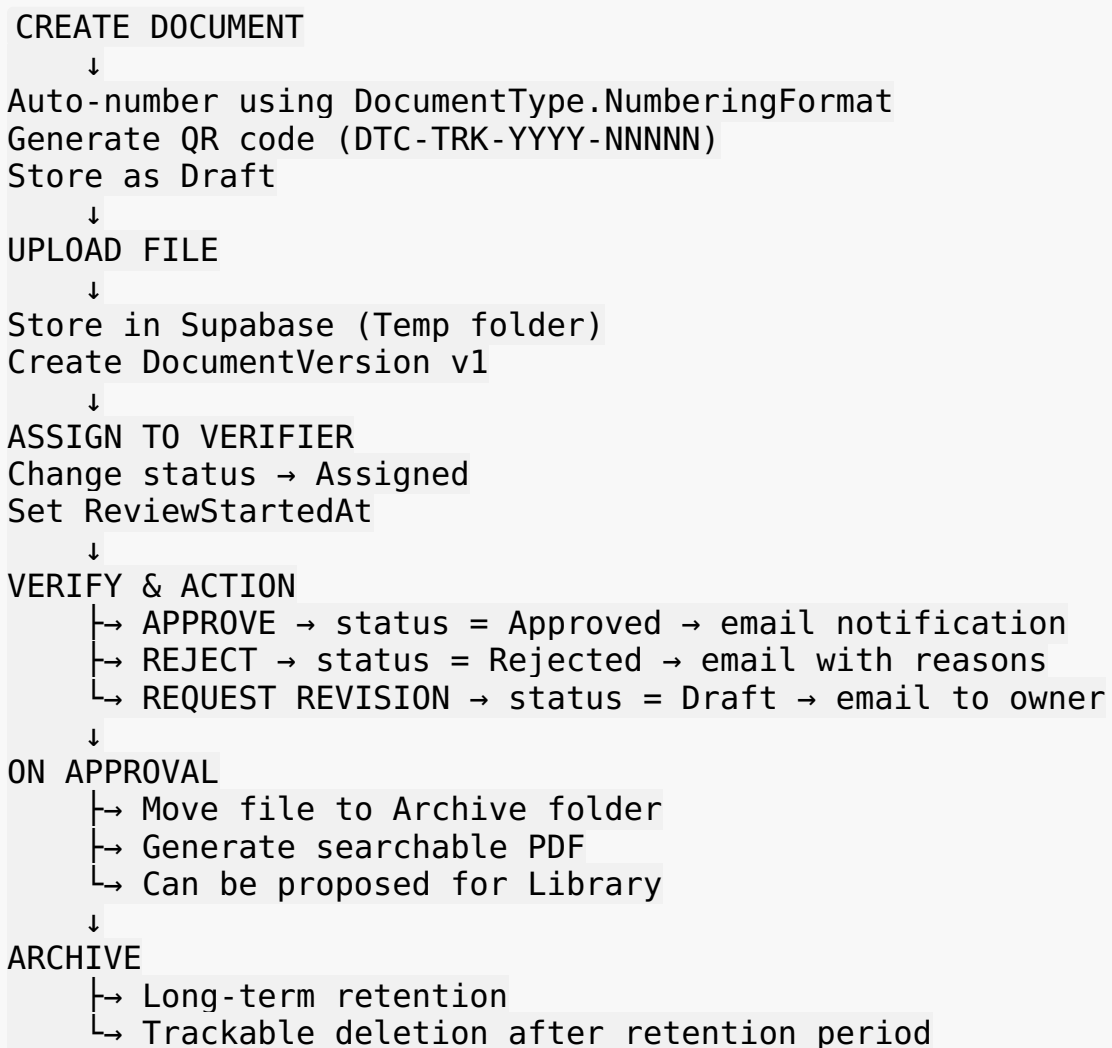
9.5 SLA Configuration

Default SLA thresholds (configurable via SystemSetting):

Transition	Threshold	Alert
Submitted → Received	4 hours	Email to assigned user
Received → UnderReview	8 hours	Email to assigned user
UnderReview → Approved/ Rejected	24 hours	Email to manager
Vendor Submission (pending)	30 days	Auto-expire

10. KEY WORKFLOWS & PROCESSES

10.1 Internal Document Workflow



10.2 Vendor Submission Workflow

VENDOR REGISTERS



Email + Password + Company info
Create User with role = "Vendor"



SUBMIT DOCUMENT



Create PendingVendorRequest (auto-numbered: SUB-YYYY-NNNNN)
Upload PDF file ≤ 100MB
Valid formats: PDF only
Files stored in: vendors/{vendorId}/{submissionId}/



ENQUEUE FOR ANALYSIS



Hangfire background job: AnalysisJob.ProcessAsync()



ANALYSIS PHASE

- ↳ Download PDF from Supabase Storage
- ↳ Send to OCR microservice
- ↳ Check DPI (must be ≥ 300)
 - ↳ If fail: Auto-reject with "Kualitas Scan Tidak Memadai"
- ↳ Classify document type (INVOICE, SPPP, etc.)
- ↳ Extract fields using regex patterns
- ↳ Detect signatory name
- ↳ Generate AI grade (Excellent/Good/Fair/Poor)
- ↳ Calculate AI score (1-10)
- ↳ Generate searchable PDF
- ↳ Create audit trail



IF ANALYSIS FAILED

- ↳ status = Rejected, AiGrade = Invalid
- ↳ Email vendor: "Analysis failed"
- ↳ Vendor cannot resubmit same file



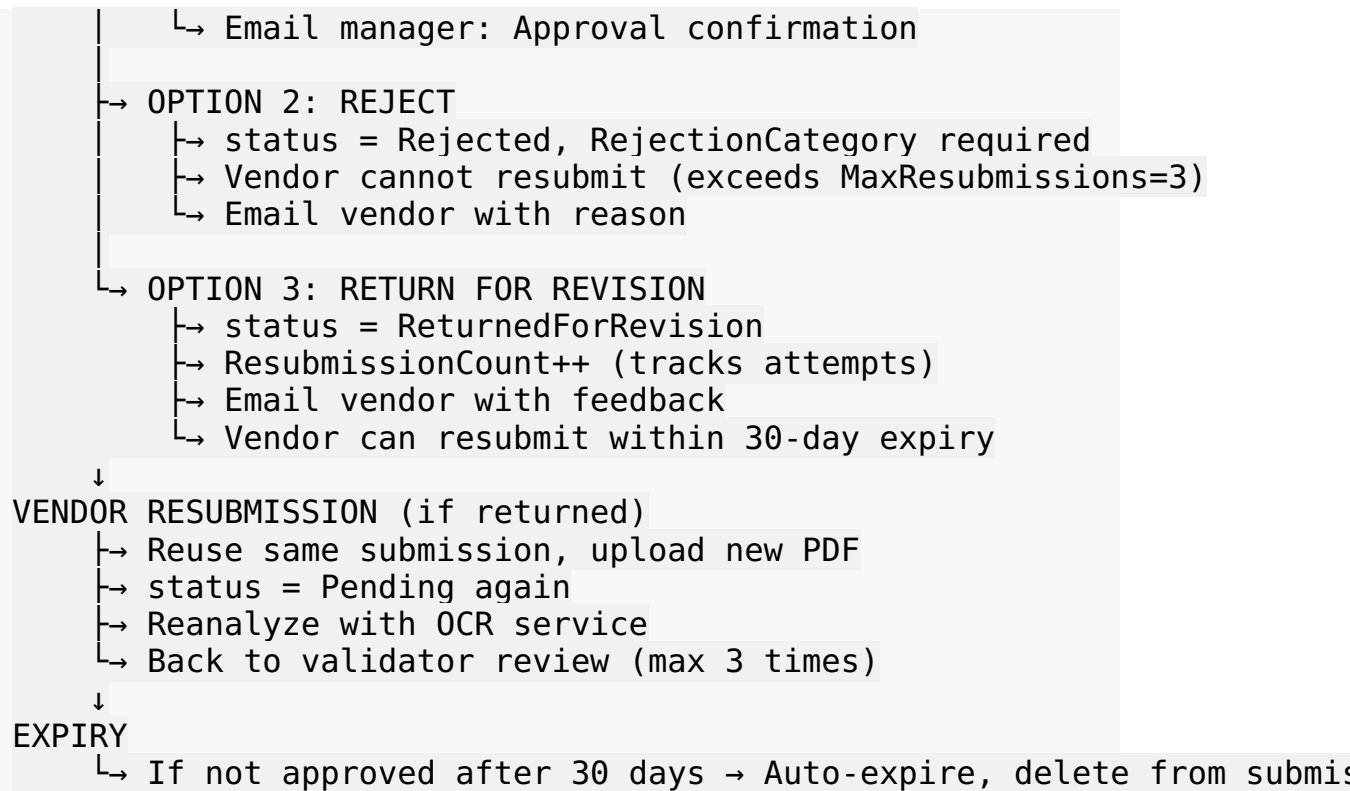
IF ANALYSIS SUCCEEDED

- ↳ status = Ready For Review, AiGrade = Excellent|Good|Fair|Poor
- ↳ Email validator: "New submission ready"
- ↳ Submission appears in validator queue

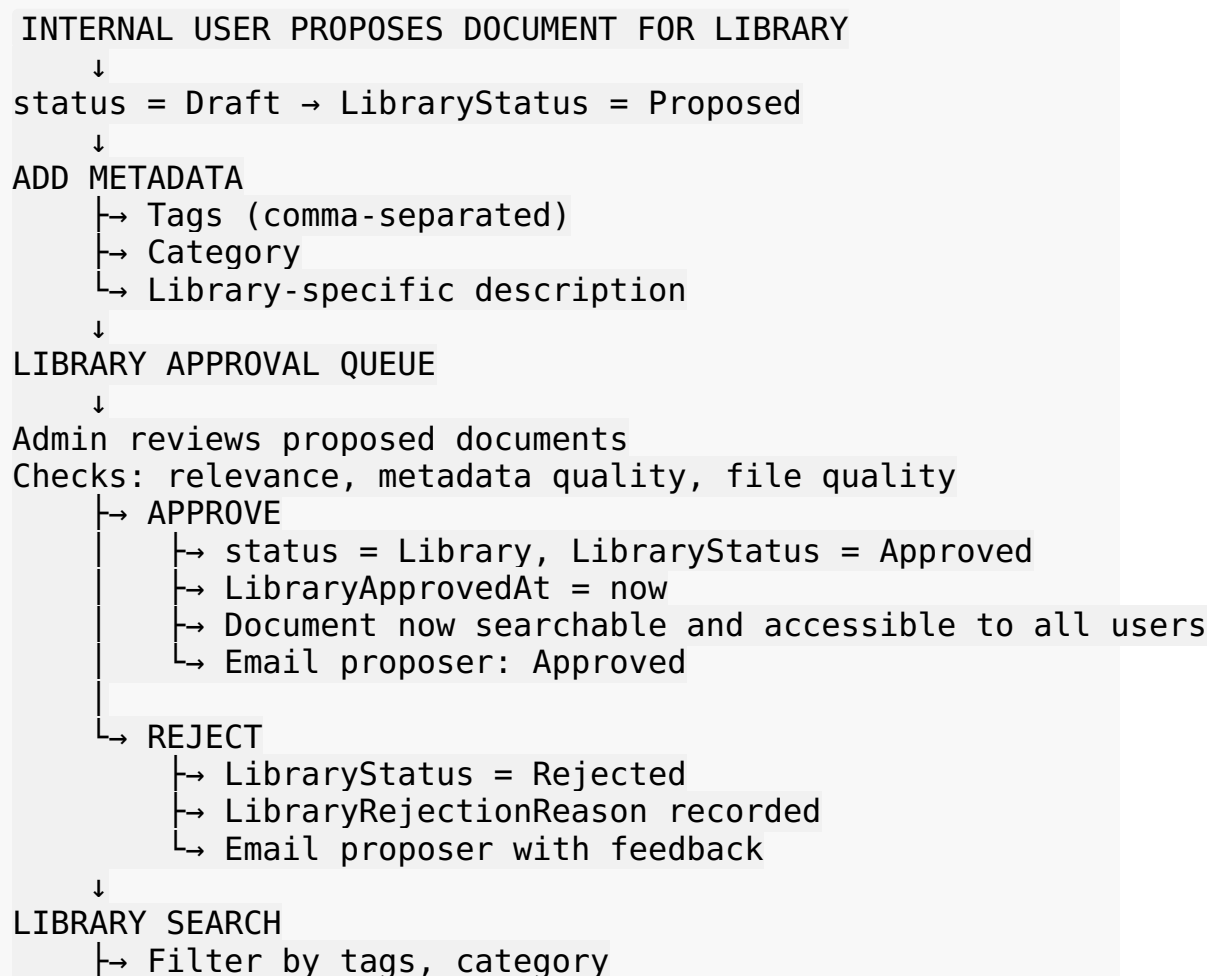


VALIDATOR REVIEW

- ↳ View analyzed document with extracted fields
- ↳ View OCR confidence scores and AI assessment
- ↳ Review DPI check result
- ↳ OPTION 1: APPROVE
 - ↳ status = Approved
 - ↳ Create Document with auto-number
 - ↳ Link to PendingVendorRequest (ResultDocumentId)
 - ↳ Email vendor: "Approved, document number is XYZ"



10.3 Library Management



- ↳ Full-text search on title, description
- ↳ Version history accessible

10.4 Physical Document Handover Tracking

DOCUMENT ARRIVES AT FRONT DESK

↓

Scan QR code: DTC-TRK-YYYY-NNNNN

- ↳ Lookup Document in system
- ↳ status → Received
- ↳ Add tracking event: Received
- ↳ ReceivedAt = timestamp

↓

ASSIGN TO VERIFIER

- ↳ status → Assigned
- ↳ AssignedToUserId = verifier ID
- ↳ Email verifier: "Document assigned to you"

↓

VERIFIER PICKS UP DOCUMENT

- ↳ status → UnderReview
- ↳ ReviewStartedAt = timestamp
- ↳ Add tracking event: ReviewStarted

↓

REVIEW COMPLETE

↳ APPROVE

- ↳ status → Approved
- ↳ Add tracking event: Approved
- ↳ Email submitter: "Document approved"
- ↳ Email submitter: "Document approved"

↳ RETURN TO SUBMITTER

- ↳ Generate OTP (6-digit code)
- ↳ OTP sent via email + SMS
- ↳ Submitter scans QR + enters OTP
- ↳ OtpConfirmedAt confirms receipt
- ↳ status → Returned
- ↳ Add tracking event: Returned
- ↳ If submitter absent: upload photo proof
- ↳ Email submitter: "Document returned" + OTP

↓

TRACKING LOG

- ↳ Full audit trail: who, what, when, IP address

11. SECURITY FEATURES

11.1 Authentication & Authorization

- JWT with HS512 algorithm (64-byte key)

- Refresh token rotation to prevent token theft
- Token revocation support
- Role-based access control (5 roles)
- Soft delete for data retention compliance

11.2 Data Protection

- PostgreSQL encrypted connections (SSL required)
- Supabase service key for server-to-server access
- System settings support encrypted fields
- Passwords hashed with industry-standard algorithms
- Environment variables for secrets (no hardcoded credentials)

11.3 API Security

- CORS configuration (AllowedHosts: "*" in appsettings)
- Global error handling (no stack traces in responses)
- Request validation on all inputs
- Rate limiting (configurable via middleware)
- Logger filters for sensitive data

11.4 File Upload Security

- File type validation (.pdf only)
- File size limits (1KB - 100MB)
- Virus scanning (configurable)
- Quarantine folder for verification
- Automatic cleanup of temp files

11.5 DPI Quality Enforcement

- Minimum 300 DPI requirement for vendor documents
 - Auto-reject if DPI too low
 - Detected DPI stored for compliance audit
-

12. PERFORMANCE & SCALABILITY

12.1 Database Optimization

- Use indexes on frequently searched columns: Email, DocumentNumber, Status
- Connection pooling via Supabase
- Query filtering with `skip/take` for pagination
- Global query filters reduce queries on soft-deleted items

12.2 Background Job Processing

- Hangfire with PostgreSQL persistence (survives restarts)
- 2 worker threads (configurable for production)
- Recurring jobs for SLA monitoring and cleanup
- Long timeouts (20 minutes) for large PDF analysis

12.3 Microservice Separation

- OCR service runs independently (separate Python process)
- Horizontal scaling: run multiple OCR instances + load balancer
- Async job queue prevents blocking on OCR requests
- Graceful degradation if OCR service down

12.4 File Storage

- Supabase provides CDN for document downloads
- Temporary files cleaned up periodically
- Archive tier for long-term retention
- S3-compatible API for multi-region replication

12.5 Caching Opportunities

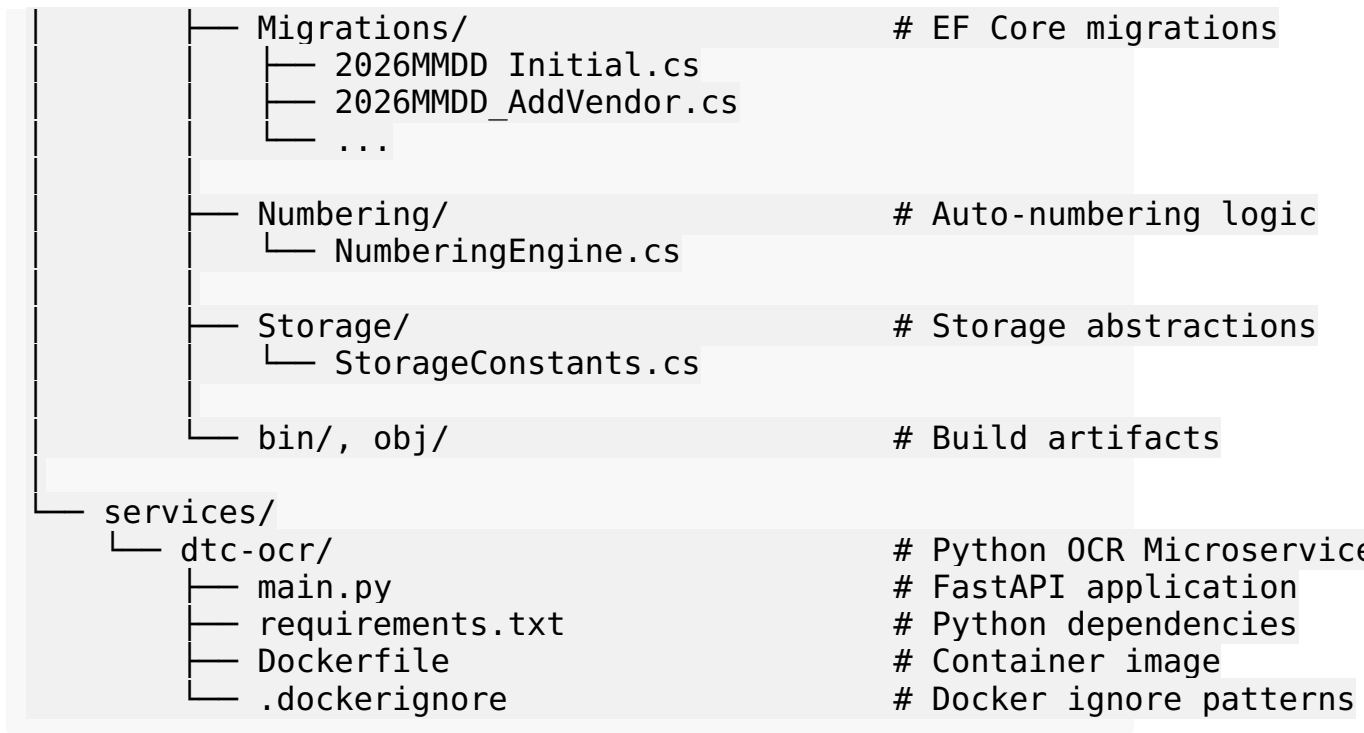
- Cache DocumentTypes (rarely change)
 - Cache SystemSettings (frequently accessed)
 - Cache user roles in JWT (no per-request lookup)
 - Cache organization functions
-

13. FILE STRUCTURE & ORGANIZATION

```
/workspaces/dtc-sys/
├── dev.sh                # Development CLI
├── Dtc.slnx              # Solution file
├── README.md             # Project docs
├── src/
│   ├── Dtc.Api/          # API Layer
│   │   ├── Program.cs    # Startup & configuration
│   │   ├── appsettings.json # Production settings
│   │   ├── appsettings.Development.json # Dev settings
│   │   ├── Dtc.Api.csproj # Project file
│   │   ├── Dtc.Api.http  # REST client test file
│   │   └── Controllers/  # API endpoints
│   │       └── AuthController.cs
│   └── ...
```

- DocumentsController.cs
 - DocumentTypesController.cs
 - LibraryController.cs
 - OrgFunctionsController.cs
 - SystemSettingsController.cs
 - TrackingController.cs
 - UsersController.cs
 - ValidatorController.cs
 - VendorController.cs
- Middleware/
 - ErrorHandlingMiddleware.cs # Global exception handler
- Properties/
 - launchSettings.json # Debug profiles
- bin/, obj/ # Build artifacts
- Dtc.Application/ # Application Layer
 - Dtc.Application.csproj
 - DTOs/ # Data Transfer Objects
 - AuthDtos.cs
 - DocumentDtos.cs
 - DocumentTypeDtos.cs
 - LibraryDtos.cs
 - OrgFunctionDtos.cs
 - SystemSettingDtos.cs
 - TrackingDtos.cs
 - UserDtos.cs
 - VendorDtos.cs
 - Interfaces/ # Service contracts
 - IAuthService.cs
 - IDocumentService.cs
 - IDocumentTypeService.cs
 - IEmailService.cs
 - ILibraryService.cs
 - IOcrService.cs
 - IOrgFunctionService.cs
 - IQRCodeService.cs
 - IStorageService.cs
 - ISystemSettingService.cs
 - ITrackingService.cs
 - IUserService.cs
 - IValidatorService.cs
 - IVendorService.cs
 - bin/, obj/ # Build artifacts
- Dtc.Domain/ # Domain/Entity Layer
 - Dtc.Domain.csproj

- Common/
 - BaseEntity.cs # Base class for all entities
 - Roles.cs # Role constants
- Entities/ # Database models
 - Document.cs
 - DocumentTracking.cs
 - DocumentType.cs
 - DocumentVersion.cs
 - OrganizationFunction.cs
 - PendingVendorRequest.cs
 - SignatoryConfig.cs
 - SystemSetting.cs
 - User.cs
 - Workflow.cs
 - WorkflowInstance.cs
- Enums/
 - DocumentEnums.cs # Comprehensive enum definitions
- bin/, obj/ # Build artifacts
- Dtc.Infrastructure/ # Infrastructure/Persistence
 - Dtc.Infrastructure.csproj
 - DependencyInjection.cs # Service registration
 - Persistence/
 - DtcDbContext.cs # EF Core DbContext
 - DbSeeder.cs # Database initialization
 - Configurations/ # EF model mappings
 - Services/ # Service implementations
 - AuthService.cs
 - DocumentService.cs
 - DocumentTypeService.cs
 - EmailService.cs
 - LibraryService.cs
 - OcrService.cs # OCR microservice client
 - OrgFunctionService.cs
 - QrCodeService.cs
 - SupabaseStorageService.cs # S3-compatible storage
 - SystemSettingService.cs
 - TrackingService.cs
 - UserService.cs
 - ValidatorService.cs
 - VendorService.cs
 - Jobs/ # Hangfire background jobs
 - AnalysisJob.cs # OCR analysis processor
 - SlaAlertJob.cs # SLA monitoring



14. KEY BUSINESS RULES & VALIDATIONS

14.1 Document Numbering

- Format per DocumentType: e.g. INV/{YEAR}/{DEPT}/{SEQ}
- Variables:
 - {YEAR} : Current year (4 digits)
 - {DEPT} : Organization function code
 - {SEQ} : Auto-incrementing sequence (zero-padded)
- Example: INV/2026/PKRMng/00042 for invoice from HR department
- Scope per: DocumentType + OrganizationFunction + Year combination
- Sequence restarts each year

14.2 Document Status Transitions

Draft	→	[Submitted OR Archived]
Submitted	→	[Received OR Archived]
Received	→	[Assigned OR Archived]
Assigned	→	[UnderReview OR Archived]
UnderReview	→	[Approved, Returned, OR Rejected]
Approved	→	[Archived]
Returned	→	[Submitted OR Archived]
Rejected	→	[Archived]
Archived	→	[DeletedPermanently] # After retention period

14.3 User Roles & Permissions

Role	Create Doc	Edit Doc	Upload File	Review Submissions	Approve Vendor	Manage Users
SysAdmin	✓	✓	✓	✓	✓	✓
Admin	✓	✓	✓	✓	✓	✓
Validator	-	-	-	✓	✓	-
User	✓	own	✓	-	-	-
Vendor	-	-	✓	-	-	-

14.4 SLA Rules

- Document must be received within **4 hours** of submission
- Review must start within **8 hours** of receipt
- Complete review within **24 hours** of start
- Alert escalation to manager if breached

14.5 Vendor Submission Rules

- Maximum file size: **100 MB**
- Minimum file size: **1 KB**
- Allowed formats: **PDF only**
- Minimum DPI: **300 DPI** (auto-reject if lower)
- Auto-expiry: **30 days** from creation
- Max resubmissions: **3 attempts** after rejection

14.6 Library Requirements

- Proposed document requires library-specific metadata:
 - Tags (recommended 3-5)
 - Category (required)
 - Description (required)
 - Searchable PDF generation required
 - Version history maintained
-

15. THIRD-PARTY INTEGRATIONS & EXTERNAL SERVICES

15.1 Supabase

- **Database:** PostgreSQL hosting with connection pooling
- **Storage:** S3-compatible bucket for document files
- **Auth:** JWT-based authentication (optional; using custom in this project)
- **Status:** Production deployment

15.2 Email Service

- SMTP configuration (provider not specified in config)
- Notifications:
 - User registration confirmations
 - Vendor submission status (approved/rejected/returned)
 - SLA alerts for overdue documents
 - System notifications

15.3 OCR Service

- Custom FastAPI microservice (internal)
 - Deployment: Same server or separate container
 - Communication: HTTP multipart form uploads
 - Timeout: 20 minutes for large PDF analysis
-

16. TESTING & DEBUGGING

16.1 API Testing

- [Dtc.Api.http](#) - REST Client test file
- Contains sample requests for all endpoints
- Can be executed directly in VS Code with REST Client extension

16.2 Swagger/OpenAPI

- Interactive API documentation at `/swagger/ui`
- Try-it-out functionality for testing
- Request schema validation
- Response schema documentation

16.3 Database Debugging

- EF Core logging enabled in appsettings
- SQL queries visible in debug output
- Hangfire dashboard for job monitoring at `/hangfire` (dev only)

16.4 Health Check

- Endpoint: `GET /health`
 - Validates basic system connectivity
 - Used by load balancers for health monitoring
-

17. FUTURE ENHANCEMENTS & EXTENSIBILITY

17.1 Planned Features

- Advanced document search with Elasticsearch
- Multi-language OCR support
- Barcode/QR code generation for physical tracking
- Digital signature integration
- Document conversion (PDF ↔ Word ↔ Excel)
- Batch import/export capabilities

17.2 Scalability Improvements

- Microservice extraction: Separate notification service
- Cache layer (Redis) for frequently accessed data
- Message queue (RabbitMQ) for async processing
- CDN integration for document delivery

17.3 Integration Points

- LDAP/Active Directory for enterprise authentication
 - S3-compatible storage (AWS, MinIO, etc.)
 - Email service providers (SendGrid, Mailgun)
 - Document management APIs (OnBase, SharePoint)
-

18. TROUBLESHOOTING & COMMON ISSUES

Issue: OCR Service Connection Failed

Solution: - Verify OCR service is running on `http://localhost:8000` - Check `OcrService:BaseUrl` in appsettings - Review error logs in `Infrastructure.Services.OcrService`

Issue: Database Migrations Fail

Solution:

```
./dev migrate                # Try applying migrations
./dev migrations-list         # Check migration status
dotnet ef database drop      # Dangerous! Resets database
./dev migrate                # Reapply all migrations
```

Issue: Vendor Submission Auto-Rejected (DPI Too Low)

Solution: - Vendor must rescan document with minimum 300 DPI - Check captured DPI in submission details - Admin can adjust minimum via `SystemSetting`

Issue: JWT Token Expired

Solution: - Use refresh token endpoint: `POST /api/auth/refresh` - Implement token refresh logic on client side - Check token expiry in `appsettings.json` (`ExpiryMinutes`)

19. DEPLOYMENT CHECKLIST

- [] Update `appsettings.json` with production Supabase credentials
- [] Set strong JWT key (64+ bytes)
- [] Configure OCR service URL or deploy OCR microservice
- [] Set up PostgreSQL backups
- [] Configure SSL certificates for HTTPS
- [] Set up log aggregation (ELK, Splunk, etc.)
- [] Configure monitoring and alerting
- [] Run database migrations: `./dev migrate`
- [] Test health endpoint: `curl http://localhost:5000/health`
- [] Load test with documentation workflow

- [] Verify background jobs with Hangfire dashboard
 - [] Performance test OCR service with various PDF types
-

20. SUMMARY OF KEY STATISTICS

Metric	Count
API Controllers	10
API Endpoints	50+
Service Interfaces	14
Service Implementations	14
Domain Entities	16
Enums	8
DTOs	40+
Database Tables	16
Roles	5
Background Jobs	2
Python Microservices	1
NuGet Packages	5 major dependencies
Python Dependencies	11
Max Document Size	100 MB
Min DPI Requirement	300 DPI
Auth Token Expiry	60 minutes
Vendor Submission Expiry	30 days
Max Resubmissions	3 attempts

This comprehensive summary covers the entire DTC-SYS architecture, including all layers, workflows, technologies, and operational details. The project demonstrates clean architecture principles with clear separation of concerns, making it maintainable and scalable for future enhancements.